

Groupe 1

Bloc 2 TD 2 - Langage PL/pgSQL

Contexte	2
Mise en place de la base HHB-Direct	2
Exercice 1	8
Exercice 2	12
Exercice 3	13
Exercice 4	14
Exercice 5	16
Exercice 6	18
Exercice 7	20
Exercice 8	22
Conclusion	24

Contexte

Ce TD nous fait découvrir le langage **PL/pgSQL**, qui permet d'écrire des **fonctions** directement dans **PostgreSQL** avec des variables, des conditions et des boucles. On travaille sur la base **HHB-Direct**, la base d'une banque en ligne avec ses **clients**, ses **comptes** et leurs **opérations**.

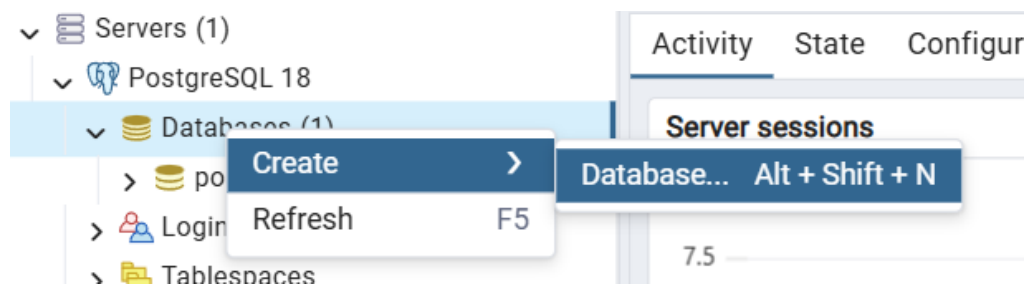
*Toutes les fonctions ont été écrites et testées dans **pgAdmin 4** avec **PostgreSQL 18**.*

*(La **BDD** est disponible sur le portfolio)*

Mise en place de la base HHB-Direct

Avant de commencer, il faut remettre en place la base **HHB-Direct**. Le fichier **hnb_direct_v1.sql** n'est pas un script SQL mais une **sauvegarde** faite avec **pg_dump** (format "custom"), il faut donc la **restaurer** et pas l'exécuter dans l'éditeur.

Dans **pgAdmin**, on crée d'abord une base vide : clic droit sur **Databases** puis **Create -> Database...** :

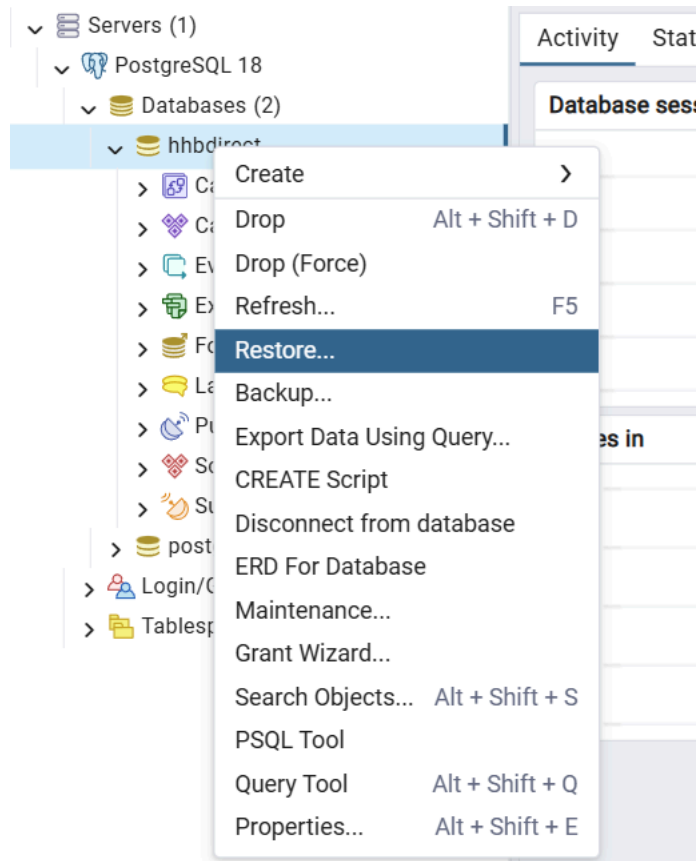


On lui donne le nom **hhbdirect** et on clique sur **Save** :

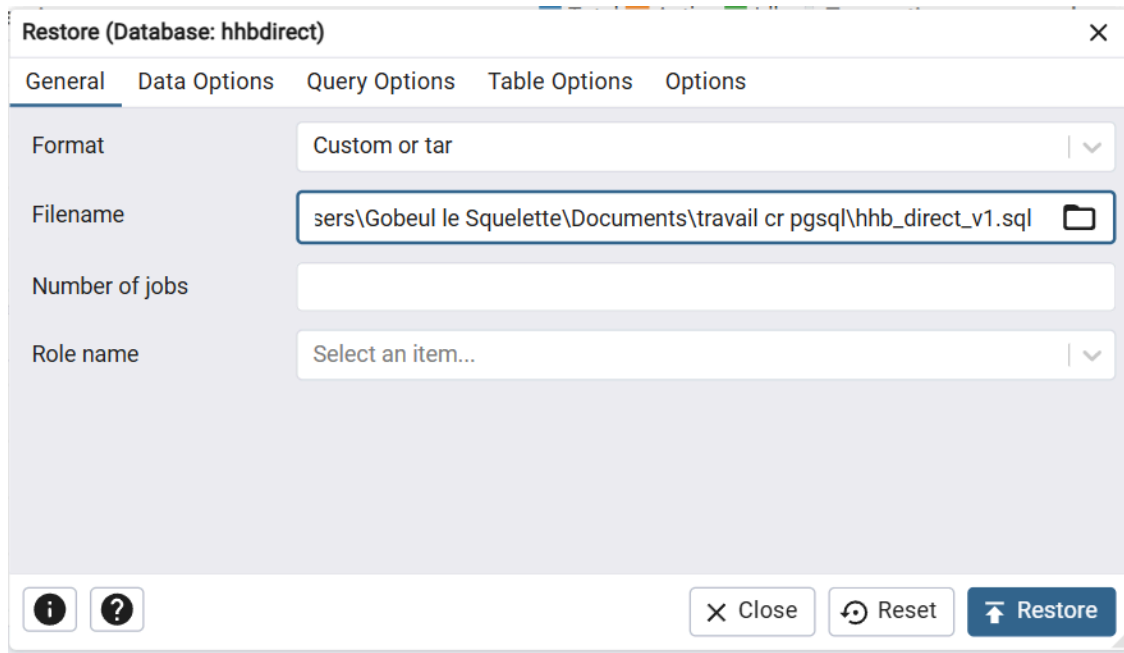
The image shows a 'Create - Database' dialog box with the following fields and controls:

- Database:** Text input field containing 'hhbdirect'.
- OID:** Empty text input field.
- Owner:** Dropdown menu showing 'postgres'.
- Comment:** Large empty text area.
- Buttons:** 'Close', 'Reset', and 'Save' (highlighted in blue) at the bottom right.

Ensuite clic droit sur la nouvelle base **hhbdirect** -> **Restore...** :



On laisse le format **Custom or tar**, on indique le chemin du fichier **hnb_direct_v1.sql** dans **Filename** puis on clique sur **Restore** :



Restore (Database: hnbdirect)

General Data Options Query Options Table Options Options

Format Custom or tar

Filename sers\Gobeul le Squelette\Documents\travail cr pgsql\hnb_direct_v1.sql

Number of jobs

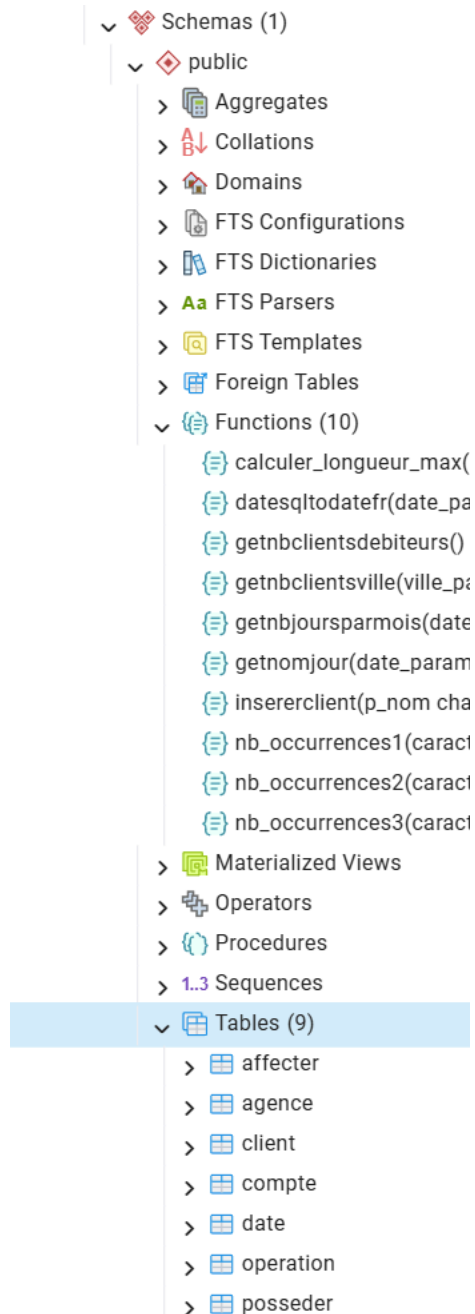
Role name Select an item...

Close Reset Restore

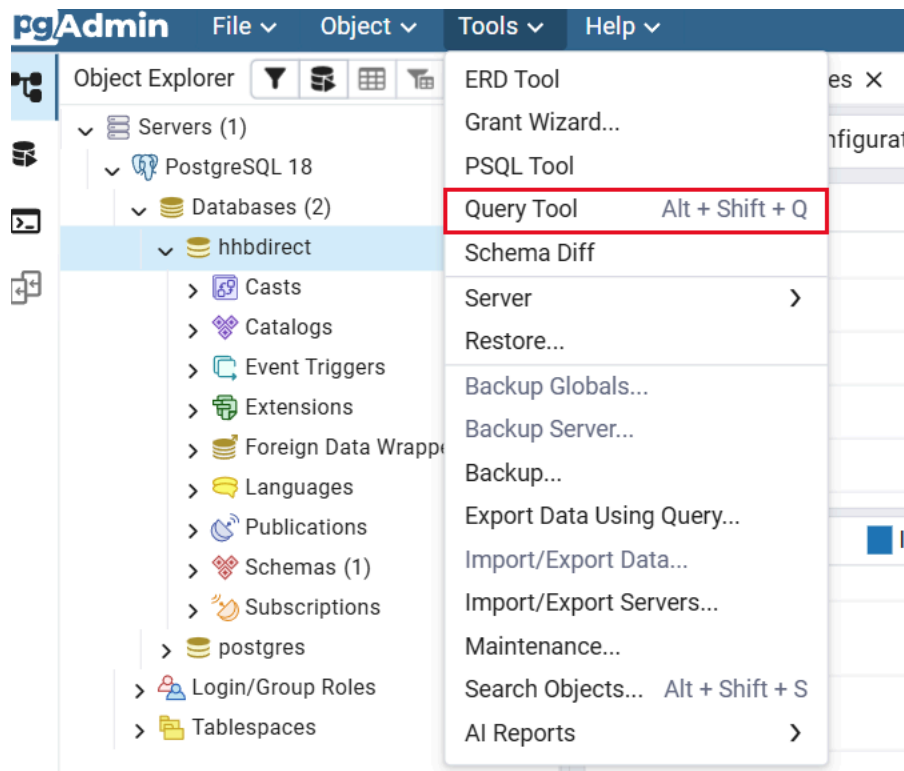
La restauration se fait en tâche de fond, on peut suivre son état dans l'onglet **Processes**, elle est bien terminée (**Finished**) :

Type	Server	Object	Start Time ▾	Status
Restore	PostgreSQL 18 (local...)	hnbdirect	18/09/2026 14:50:15	Finished

Après un rafraîchissement (F5 sur la base), on retrouve les **9 tables** dans **Schemas**
-> **public** -> **Tables**. La sauvegarde contient aussi les **10 fonctions** faites au **TD1**
(calculer_longueur_max, getnbclientsville, insererclient...):



Pour écrire les fonctions, on sélectionne la base **hhbdirect** et on ouvre l'outil de requête avec **Tools -> Query Tool** :



On regarde le contenu de la table **client** avec **`select * from client;`**, c'est elle qu'on va le plus utilisé. Il y a **6 clients** (Paul ARMANT a été ajouté au TD1) :

	num_client [PK] integer	nom_client character varying (30)	prenom_client character varying (30)	adresse_client character varying (192)	identifiant_internet character varying (30)	mdp_internet character varying (30)
1	1	DUPONT	Pierre	5 Rue du Port, 22300 LANNION	Pdupont	Pdupont
2	2	DUPONT	Annie	5 Rue du Port, 22300 LANNION	Adupont	Adupont
3	3	DELAVAL	Jean	12 Bd de l'Orne, 14234 OUISTREHAM	Jdelaval	Jdelaval
4	4	HANOT	Eric	13 Avenue de Neuilly, 94230 NOGENT SUR MAR...	Ehanot	Ehanot
5	5	LEVY	Sarah	1 Rue Neuve, 14110 CONDE SUR NOIREAU	Slevy	Slevy
6	6	ARMANT	Paul	12 Rue des Lilas, 75000 PARIS	Parmant	azerty

Exercice 1

Exécuter les fonctions suivantes et indiquer le résultat obtenu.

En copiant le code du TD tel quel dans pgAdmin, on a directement une erreur :

```
ERROR: syntax error at or near "//"  
LINE 1: // getonetuple()  
      ^
```

```
ERREUR: syntax error at or near "//"  
SQL state: 42601  
Character: 1
```

En SQL les commentaires ne s'écrivent pas avec `//` mais avec `--`, on remplace donc les `//` par des `--` dans les 3 fonctions.

Première fonction, **getOneTuple()** :

```
1  -- getonetuple()  
2  CREATE OR REPLACE FUNCTION getOneTuple() RETURNS record  
3  LANGUAGE plpgsql  
4  AS  
5  $function$  
6      declare  
7          res record ;  
8  ▾  begin  
9          select into res * from client ;  
10         return res ;  
11     end;  
12  $function$ ;
```

La fonction est bien créée :

Data Output Messages Notifications

CREATE FUNCTION

Query returned successfully in 73 msec.

On l'exécute avec `select getonetuple();` :

getonetuple record		
1	1,DUPONT,Pierre,5 Rue du Port, 22300 LANNION,Pdupont,Pdupont	

Résultat : la fonction renvoie **une seule ligne**, le premier client (**Pierre DUPONT**), le tout dans une seule colonne de type **record**. Le **SELECT INTO** ne garde que la première ligne de la requête, même si la table client en contient 6.

Si on essaye de l'utiliser comme une table avec `select * from getonetuple();`, on a une erreur :

```
ERROR:  une liste de définition de colonnes est requise pour les fonctions renvoyant un « record »
LINE 1: select * from getonetuple();
                  ^
```

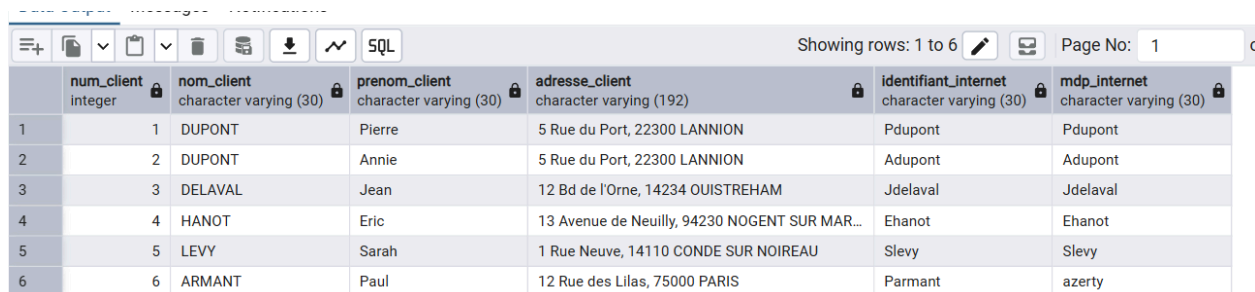
```
ERREUR:  une liste de définition de colonnes est requise pour les fonctions renvoyant un « record »
SQL state: 42601
Character: 15
```

PostgreSQL ne connaît pas les colonnes d'un **record**, il faudrait lui préciser la liste des colonnes et leurs types dans la requête.

Deuxième fonction, **getOneTupleTable()** :

```
1  -- getonetupletable()
2  CREATE OR REPLACE FUNCTION getOneTupleTable()
3  RETURNS SETOF client
4  LANGUAGE plpgsql
5  AS $function$
6  begin
7      return query select * from client ;
8  end;
9  $function$ ;
```

On l'exécute avec **select * from getonetupletable();** :



	num_client integer	nom_client character varying (30)	prenom_client character varying (30)	adresse_client character varying (192)	identifiant_Internet character varying (30)	mdp_internet character varying (30)
1	1	DUPONT	Pierre	5 Rue du Port, 22300 LANNION	Pdupont	Pdupont
2	2	DUPONT	Annie	5 Rue du Port, 22300 LANNION	Adupont	Adupont
3	3	DELAVAL	Jean	12 Bd de l'Orne, 14234 OUISTREHAM	Jdelaval	Jdelaval
4	4	HANOT	Eric	13 Avenue de Neuilly, 94230 NOGENT SUR MAR...	Ehanot	Ehanot
5	5	LEVY	Sarah	1 Rue Neuve, 14110 CONDE SUR NOIREAU	Slevy	Slevy
6	6	ARMANT	Paul	12 Rue des Lilas, 75000 PARIS	Parmant	azerty

Résultat : cette fois on récupère **tous les clients** (6 lignes) avec les vraies colonnes de la table. La fonction renvoie un **SETOF client** (un ensemble de lignes de la table client) et **RETURN QUERY** renvoie directement toutes les lignes de la requête.

Troisième fonction, **lesClients()** :

```
1  -- lesclients()
2  CREATE OR REPLACE FUNCTION lesClients() RETURNS SETOF client
3  LANGUAGE plpgsql
4  AS $function$
5  declare
6      res client%ROWTYPE ;
7  begin
8      for res in select * from client
9      loop
10         return next res ;
11     end loop;
12     return;
13 end ;
14 $function$ ;
```

Et `select * from lesclients();` :

	num_client integer	nom_client character varying (30)	prenom_client character varying (30)	adresse_client character varying (192)	identifiant_internet character varying (30)	mdp_internet character varying
1	1	DUPONT	Pierre	5 Rue du Port, 22300 LANNION	Pdupont	Pdupont
2	2	DUPONT	Annie	5 Rue du Port, 22300 LANNION	Adupont	Adupont
3	3	DELAVAL	Jean	12 Bd de l'Orne, 14234 OUISTREHAM	Jdelaval	Jdelaval
4	4	HANOT	Eric	13 Avenue de Neuilly, 94230 NOGENT SUR MAR...	Ehanot	Ehanot
5	5	LEVY	Sarah	1 Rue Neuve, 14110 CONDE SUR NOIREAU	Slevy	Slevy
6	6	ARMANT	Paul	12 Rue des Lilas, 75000 PARIS	Parmant	azerty

Résultat : on obtient exactement la même chose qu'avec `getOneTupleTable()`. La différence est dans la façon de faire : ici la fonction parcourt les clients un par un avec une boucle **FOR**, la variable **res** est de type **client%ROWTYPE** (une ligne de la table client), chaque ligne est ajoutée au résultat avec **RETURN NEXT** et le **RETURN** à la fin termine la fonction.

Exercice 2

Écrire une fonction qui retourne le nombre de clients habitant dans une ville. Le nom de la ville est un paramètre de la fonction.

La table **client** n'a pas de colonne ville, la ville se trouve à la fin de **adresse_client** (par exemple "5 Rue du Port, 22300 **LANNION**"). On utilise donc un **LIKE** avec un % devant le nom de la ville, et **upper()** des deux côtés pour ne pas être embêté par les majuscules/minuscules :

```
1 CREATE OR REPLACE FUNCTION nb_clients_ville(p_ville varchar) RETURNS integer
2 LANGUAGE plpgsql
3 AS $function$
4     declare
5         nb integer ;
6     begin
7         select count(*) into nb
8         from client
9         where upper(adresse_client) like '%' || upper(p_ville) ;
10        return nb ;
11    end;
12 $function$ ;
```

On teste avec 3 villes en même temps :

```
1 select nb_clients_ville('Lannion') as lannion,
2        nb_clients_ville('PARIS') as paris,
3        nb_clients_ville('Caen') as caen;
```

	lannion integer	paris integer	caen integer
1	2	1	0

On a bien **2** clients à Lannion (Pierre et Annie DUPONT), **1** à Paris (Paul ARMANT) et **0** à Caen, ce qui correspond à la table client.

Exercice 3

Écrire une fonction qui retourne le nombre de clients débiteurs.

Un client est **débiteur** s'il a au moins un compte avec un **solde négatif**. Les comptes sont reliés aux clients par la table **posseder**, donc on fait une jointure entre **posseder** et **compte** (sur **id_type** et **num_compte**, la clé du compte). Le **count(distinct ...)** évite de compter 2 fois un client qui aurait plusieurs comptes dans le rouge :

```
1 CREATE OR REPLACE FUNCTION nb_clients_debiteurs() RETURNS integer
2 LANGUAGE plpgsql
3 AS $function$
4     declare
5         nb integer ;
6     begin
7         select count(distinct p.num_client) into nb
8         from posseder p
9         inner join compte c on c.id_type = p.id_type and c.num_compte = p.num_compte
10        where c.solde < 0 ;
11        return nb ;
12    end;
13 $function$ ;
```

select nb_clients_debiteurs(); renvoie :

nb_clients_debiteurs	
integer	
1	1

On vérifie avec une requête qui affiche les clients concernés :

```
1 select c.nom_client, c.prenom_client, co.id_type, co.num_compte, co.solde
2 from client c
3 inner join posseder p on p.num_client = c.num_client
4 inner join compte co on co.id_type = p.id_type and co.num_compte = p.num_compte
5 where co.solde < 0;
```

	nom_client character varying (30)	prenom_client character varying (30)	id_type smallint	num_compte integer	solde integer
1	LEVY	Sarah	1	5	-1200

C'est bien **Sarah LEVY** avec son compte courant (type 1, n°5) à **-1200**, c'est la seule cliente débitrice.

Exercice 4

Écrire une fonction qui permet d'insérer un tuple dans la table client. Les valeurs des colonnes seront fournies en argument à la fonction sauf pour le numéro du client. Le numéro du client devra être calculé dans la fonction. La fonction récupère le dernier numéro et l'incrmente pour créer le nouveau numéro du client à enregistrer. Une information devra indiquer si le tuple a été bien enregistré ou pas.

On récupère le dernier numéro avec **max(num_client)** et on ajoute 1 (le **coalesce** sert au cas où la table serait vide, max renverrait **null**). Pour l'information, la fonction renvoie un message : si l'insertion a marché on renvoie le numéro du nouveau client, et s'il y a une erreur, le bloc **EXCEPTION** l'attrape et renvoie un message d'erreur au lieu de tout faire planter :

```
1 CREATE OR REPLACE FUNCTION ajouter_client(p_nom varchar, p_prenom varchar, p_adresse varchar,
2     p_id_internet varchar, p_mdp varchar) RETURNS varchar
3 LANGUAGE plpgsql
4 AS $function$
5     declare
6         num integer ;
7     begin
8         -- on récupère le dernier numéro et on l'incrmente
9         select max(num_client) into num from client ;
10        num := coalesce(num, 0) + 1 ;
11
12        insert into client (num_client, nom_client, prenom_client, adresse_client,
13            identifiant_internet, mdp_internet)
14        values (num, p_nom, p_prenom, p_adresse, p_id_internet, p_mdp) ;
15
16        if found then
17            return 'Le client n°' || num || ' a bien été enregistré' ;
18        end if ;
19        return 'Le client n''a pas été enregistré' ;
20    exception
21        when others then
22            return 'Erreur, le client n''a pas été enregistré : ' || sqlerrm ;
23    end;
24 $function$ ;
```

On ajoute une nouvelle cliente :

```
1 select ajouter_client('MARTIN', 'Julie', '8 Rue des Lilas, 22300 LANNION', 'Jmartin', 'Jmartin');
```

	ajouter_client character varying
1	Le client n°7 a bien été enregistré

Elle apparaît bien dans la table **client** avec le numéro **7** (le dernier était 6) :

	num_client [PK] integer	nom_client character varying (30)	prenom_client character varying (30)	adresse_client character varying (192)	identifiant_internet character varying (30)	mdp_internet character varying (30)
1	1	DUPONT	Pierre	5 Rue du Port, 22300 LANNION	Pdupont	Pdupont
2	2	DUPONT	Annie	5 Rue du Port, 22300 LANNION	Adupont	Adupont
3	3	DELAVAL	Jean	12 Bd de l'Orne, 14234 OUISTREHAM	Jdelaval	Jdelaval
4	4	HANOT	Eric	13 Avenue de Neuilly, 94230 NOGENT SUR MAR...	Ehanot	Ehanot
5	5	LEVY	Sarah	1 Rue Neuve, 14110 CONDE SUR NOIREAU	Slevy	Slevy
6	6	ARMANT	Paul	12 Rue des Lilas, 75000 PARIS	Parmant	azerty
7	7	MARTIN	Julie	8 Rue des Lilas, 22300 LANNION	Jmartin	Jmartin

On teste aussi un cas où l'insertion échoue, avec un nom trop long (la colonne **nom_client** est limitée à 30 caractères) :

```
1 select ajouter_client('NOM_BEAUCOUP_TROP_LONG_POUR_LA_COLONNE', 'Test', '1 Rue du Test, 14000 CAEN', 'Ttest', 'Ttest');
```

	ajouter_client character varying
1	Erreur, le client n'a pas été enregistré : valeur trop longue pour le type character varying(30)

Le client n'est pas enregistré et la fonction l'indique avec le message d'erreur de PostgreSQL.

Au passage, la fonction de l'exercice 2 trouve maintenant **3** clients à Lannion avec Julie MARTIN (**select nb_clients_ville('Lannion');**) :

	nb_clients_ville integer
1	3

Exercice 5

Écrire une fonction nommée `nb_operation_compte_mois()` qui permet de calculer le nombre d'opérations pour un mois passé en argument pour un compte bien précis dont le numéro est passé en argument.

La fonction reçoit le **mois** (1..12), l'**année**, le **numéro** et le **type** du compte. Dans cette base, un compte est identifié par son numéro **et** son type (la clé de **compte** est **id_type + num_compte**) : la cliente n°5 a par exemple un compte courant n°5 et un livret A n°5.

Dans la table **operation**, le compte qui fait l'opération est dans **num_compte/id_type**, et le compte qui reçoit un versement est dans **num_compte_vers/id_type_vers**. Comme le TD demande de compter aussi les versements reçus, on teste les deux avec un **OR**. Pour le mois et l'année, on utilise **date_part()** :

```
1 CREATE OR REPLACE FUNCTION nb_operation_compte_mois(p_mois integer, p_annee integer,
2   p_num_compte integer, p_id_type integer) RETURNS integer
3 LANGUAGE plpgsql
4 AS $function$
5   declare
6     nb integer ;
7   begin
8     select count(*) into nb
9     from operation
10    where date_part('month', date) = p_mois
11    and date_part('year', date) = p_annee
12    and ((num_compte = p_num_compte and id_type = p_id_type)
13    or (num_compte_vers = p_num_compte and id_type_vers = p_id_type)) ;
14    return nb ;
15  end;
16 $function$ ;
```

Pour tester, on regarde d'abord les opérations des comptes n°5 (**`select * from operation where num_compte = 5;`**) :

	id_operation [PK] bigint	num_compte integer	date date	id_type smallint	id_type_vers smallint	num_compte_vers integer	designation character varying (50)	type_operation character varying (20)	montant integer
1	10	5	2012-10-27	1	[null]	[null]	Paye	CREDIT	2500
2	11	5	2012-10-27	1	2	5	Virement	VIREMENT	500
3	12	5	2012-11-01	1	[null]	[null]	Remboursement	DEBIT	3200

L'opération **11** est un **virement** du compte courant n°5 (type 1) vers le livret A n°5 (type 2), le 27/10/2012.

On teste la fonction sur plusieurs comptes :

```
1 select nb_operation_compte_mois(10, 2012, 5, 1) as courant_oct,  
2        nb_operation_compte_mois(10, 2012, 5, 2) as livret_oct,  
3        nb_operation_compte_mois(11, 2012, 4, 1) as compte4_nov;
```

	courant_oct integer	livret_oct integer	compte4_nov integer
1	2	1	3

En octobre 2012, le compte courant n°5 a **2** opérations (la paye et le virement), le livret A n°5 en a **1** (il reçoit le virement, c'est pour ça qu'il fallait tester **num_compte_vers**), et le compte courant n°4 a bien **3** opérations en novembre 2012.

Exercice 6

Écrire une fonction nommée `creer_id_internet()` qui crée l'identifiant et le mot de passe internet du client passé en argument.

La fonction reçoit le **nom** et le **prénom**. L'identifiant est la 1ère lettre du prénom en majuscule (`upper(substr(p_prenom, 1, 1))`) suivie du nom en minuscule (`lower(p_nom)`). Ensuite on compte les clients qui ont déjà ce nom et ce prénom : s'il y en a, on ajoute ce nombre à la fin (s'il y a déjà 3 André Lambert, le nouveau sera **Alambert3**). Le mot de passe étant identique à l'identifiant, la fonction renvoie cette valeur qui sert pour les deux :

```
1 CREATE OR REPLACE FUNCTION creer_id_internet(p_nom varchar, p_prenom varchar)
2 RETURNS varchar
3 LANGUAGE plpgsql
4 AS $function$
5     declare
6         id_internet varchar ;
7         nb integer ;
8     begin
9         -- 1ère lettre du prénom en majuscule + nom en minuscule
10        id_internet := upper(substr(p_prenom, 1, 1)) || lower(p_nom) ;
11
12        -- nombre de clients qui ont déjà ce nom et ce prénom
13        select count(*) into nb
14        from client
15        where upper(nom_client) = upper(p_nom)
16        and upper(prenom_client) = upper(p_prenom) ;
17
18        if nb > 0 then
19            id_internet := id_internet || nb ;
20        end if ;
21        return id_internet ;
22    end;
23 $function$ ;
```

Test avec un nouveau client et avec un client qui existe déjà (Pierre DUPONT) :

```
1 select creer_id_internet('MARTIN', 'Paul') as nouveau_client,
2      creer_id_internet('DUPONT', 'Pierre') as homonyme;
```

	nouveau_client character varying	homonyme character varying
1	Pmartin	Pdupont1

Pmartin pour Paul MARTIN qui n'existe pas encore, et **Pdupont1** car il y a déjà un Pierre DUPONT dans la base.

On peut s'en servir directement avec la fonction de l'exercice 4 pour ajouter un 2ème Pierre DUPONT :

```
1 select ajouter_client('DUPONT', 'Pierre', '3 Place du Marché, 14000 CAEN',
2      creer_id_internet('DUPONT', 'Pierre'), creer_id_internet('DUPONT', 'Pierre'));
```

	ajouter_client character varying
1	Le client n°8 a bien été enregistré

Il est enregistré avec le numéro **8**, et son identifiant et son mot de passe sont bien **Pdupont1** :

	num_client [PK] integer	nom_client character varying (30)	prenom_client character varying (30)	adresse_client character varying (192)	identifiant_internet character varying (30)	mdp_internet character varying (30)
1	1	DUPONT	Pierre	5 Rue du Port, 22300 LANNION	Pdupont	Pdupont
2	2	DUPONT	Annie	5 Rue du Port, 22300 LANNION	Adupont	Adupont
3	3	DELAVAL	Jean	12 Bd de l'Orne, 14234 OUISTREHAM	Jdelaval	Jdelaval
4	4	HANOT	Eric	13 Avenue de Neuilly, 94230 NOGENT SUR MAR...	Ehanot	Ehanot
5	5	LEVY	Sarah	1 Rue Neuve, 14110 CONDE SUR NOIREAU	Slevy	Slevy
6	6	ARMANT	Paul	12 Rue des Lilas, 75000 PARIS	Parmant	azerty
7	7	MARTIN	Julie	8 Rue des Lilas, 22300 LANNION	Jmartin	Jmartin
8	8	DUPONT	Pierre	3 Place du Marché, 14000 CAEN	Pdupont1	Pdupont1

Et si on voulait créer un 3ème Pierre DUPONT, il aurait **Pdupont2** (**select creer_id_internet('DUPONT', 'Pierre');**) :

	creer_id_internet character varying
1	Pdupont2

Exercice 7

Écrire une fonction nommée `creer_date()` qui insère dans la table `date` toutes les dates pour le mois et l'année passées en argument.

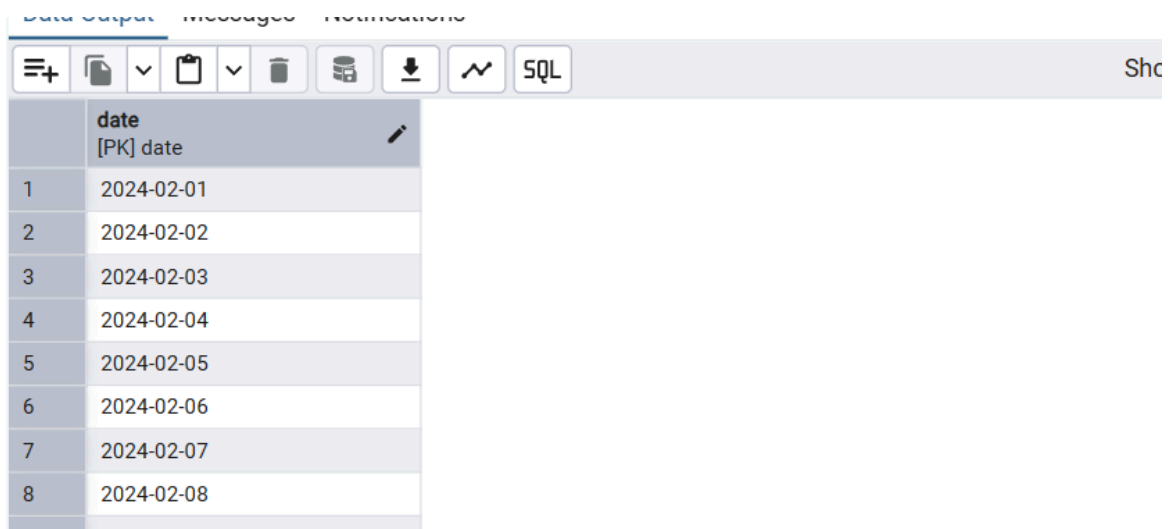
On part du 1er jour du mois avec `make_date(p_annee, p_mois, 1)`, puis on avance d'un jour à chaque tour de la boucle **WHILE** tant qu'on est toujours dans le même mois (ça gère tout seul les mois de 28, 29, 30 ou 31 jours). La colonne **date** est la clé primaire de la table, donc on vérifie avec **NOT EXISTS** que la date n'y est pas déjà avant de l'insérer. La fonction renvoie le nombre de dates insérées :

```
1 CREATE OR REPLACE FUNCTION creer_date(p_mois integer, p_annee integer)
2 RETURNS integer
3 LANGUAGE plpgsql
4 AS $function$
5     declare
6         jour date ;
7         nb integer := 0 ;
8     begin
9         jour := make_date(p_annee, p_mois, 1) ;
10        -- on avance jour par jour tant qu'on reste dans le mois
11        while date_part('month', jour) = p_mois loop
12            if not exists (select 1 from date where date.date = jour) then
13                insert into date values (jour) ;
14                nb := nb + 1 ;
15            end if ;
16            jour := jour + 1 ;
17        end loop ;
18        return nb ;
19    end;
20 $function$ ;
```

Test avec février 2024 (année bissextile) avec `select creer_date(2, 2024);`, 29 dates insérées :

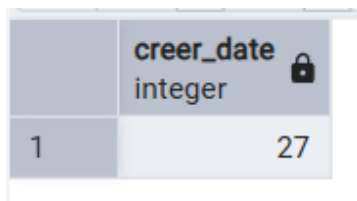
	creer_date integer
1	29

Les dates sont bien dans la table (`select * from date where date >= '2024-02-01' order by date;`, 29 lignes) :



	date [PK] date
1	2024-02-01
2	2024-02-02
3	2024-02-03
4	2024-02-04
5	2024-02-05
6	2024-02-06
7	2024-02-07
8	2024-02-08

Pour novembre 2012, la fonction n'insère que **27** dates, car le 1er, le 2 et le 3 novembre 2012 étaient déjà dans la table :



	creer_date integer
1	27

On vérifie qu'on a bien les 30 jours de novembre 2012 :

```
1 select count(*) from date
2 where date_part('month', date) = 11 and date_part('year', date) = 2012;
```



	count bigint
1	30

Exercice 8

Écrire une fonction nommée `creer_user_client()` qui crée un utilisateur PostgreSQL pour chaque client de la banque.

La fonction parcourt tous les clients avec une boucle **FOR**. Pour chaque client, on vérifie dans `pg_user` qu'il n'existe pas déjà un utilisateur avec son **identifiant_internet**, sinon on crée le rôle avec son mot de passe. Comme le nom du rôle est dans une variable, on ne peut pas écrire directement `CREATE ROLE cli.identifiant_internet` : il faut construire la requête dans une chaîne et l'exécuter avec **EXECUTE. format()** avec **%I** (nom) et **%L** (texte) ajoute les bons guillemets. On met aussi **LOGIN**, sinon le rôle ne peut pas se connecter et n'apparaît pas dans `pg_user` :

```
1 CREATE OR REPLACE FUNCTION creer_user_client() RETURNS integer
2 LANGUAGE plpgsql
3 AS $function$
4     declare
5         cli record ;
6         nb integer := 0 ;
7     begin
8         for cli in select identifiant_internet, mdp_internet from client
9             loop
10                -- on vérifie que l'utilisateur n'existe pas déjà
11                if not exists (select 1 from pg_user where username = cli.identifiant_internet) then
12                    execute format('CREATE ROLE %I LOGIN PASSWORD %L',
13                        cli.identifiant_internet, cli.mdp_internet) ;
14                    nb := nb + 1 ;
15                end if ;
16            end loop ;
17            return nb ;
18        end;
19    $function$ ;
```

Premier appel avec `select creer_user_client();`, 8 utilisateurs créés (un par client) :

	creer_user_client	
	integer	
1		8

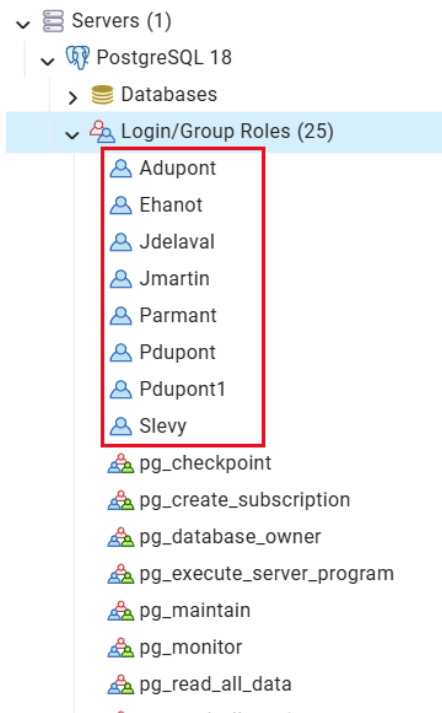
Si on relance la fonction, elle renvoie **0** car tous les utilisateurs existent déjà :

	creer_user_client	
	integer	
1		0

On les retrouve dans `pg_user` (`select username from pg_user;`). Les identifiants gardent leur majuscule (Pdupont) car %I les a mis entre guillemets, pour se connecter il faudra donc bien écrire **Pdupont** et pas pdupont :

	username name
1	postgres
2	Pdupont
3	Adupont
4	Jdelaval
5	Ehanot
6	Slevy
7	Parmant
8	Jmartin
9	Pdupont1

Et dans pgAdmin, dans **Login/Group Roles** (après un rafraîchissement) :



Conclusion

Ce TD nous a permis de découvrir le langage **PL/pgSQL** en écrivant nos propres fonctions dans **PostgreSQL** avec **pgAdmin** : des fonctions qui renvoient une valeur, un **record** ou un ensemble de lignes (**SETOF**), avec des variables, des conditions, des boucles **FOR** et **WHILE**, la gestion des erreurs avec **EXCEPTION** et même du SQL dynamique avec **EXECUTE** pour créer des utilisateurs. On a aussi vu comment restaurer une sauvegarde de base avec pgAdmin, et quelques pièges comme les commentaires en **//** dans le code du TD.

Au final on retrouve **20 fonctions** dans la base : les 10 du TD1 et les 10 de ce TD :

